

TrueUp Close — Team Product and Build Specification

1. Frozen product decision

Product name

TrueUp Close

One-sentence pitch

TrueUp Close is a policy-verified, self-improving AI close analyst embedded in an accounting team. It resolves the vendor-spend portion of month-end close by finding unbilled or inconsistent vendor expenses, gathering evidence across company systems and people, preparing supported accruals, routing only genuine uncertainty to the right person, reconciling later invoices to estimates, and improving safely from confirmed outcomes.

The problem

At month-end, companies must recognize expenses for services already received even when the vendor has not yet sent an invoice. To do this, accountants manually chase data across contracts, purchase orders, AP queues, usage systems, spreadsheets, email/Slack, prior invoices, and the general ledger.

The result is slow, error-prone close work:

- expenses may be omitted or recorded in the wrong period;
- the same obligation may be accrued twice;
- estimated amounts may be wrong;
- human reviewers spend time on routine cases;
- late invoices force manual reconciliation and explanation;
- lessons from prior errors rarely become durable controls.

What TrueUp Close owns

TrueUp Close owns the **vendor-spend accrual and exception-resolution workflow**. It does not replace the ERP, AP team, Controller, auditor, or entire accounting department.

Its objective is to take each expected vendor spend obligation to one of four valid outcomes:

Outcome	Meaning
CLOSE_READY	Amount is supported, policy-compliant, and ready for the close workflow
ACCRUED	No invoice arrived, but the system prepared a supported accrual under policy

Outcome	Meaning
RECONCILED	Later invoice was matched to a prior accrual; difference was resolved or explained
ESCALATED	Evidence is inadequate, conflicting, material, or outside policy; a named human owns the next decision

Core product doctrine

1. **Embedded teammate, not standalone tool.** The agent works with the company's existing systems and people.
2. **One shared obligation record.** The same vendor service should have one identity from contract through accrual, invoice, true-up, payment forecast, and audit evidence.
3. **Human attention is scarce.** Humans should see genuine uncertainty and material judgment calls, not routine high-confidence work.
4. **No unconstrained financial actions.** LLMs interpret evidence and propose actions; deterministic code performs calculations and validates controls.
5. **No policy bypass.** Every consequential action is checked against company procedures before it can affect ledger or workflow state.
6. **Learning is governed.** Later invoices, human resolutions, and outreach responses can create candidate policies; only replay-tested and human-approved policies become active.

2. Accounting concepts and user story

Essential terms

Term	Plain-English meaning
Vendor	Outside company that sells the business goods or services
Invoice	A vendor bill that states amount owed, service dates, and due date
AP / Accounts Payable	Team/system that processes vendor bills and schedules payment
General ledger / GL	Official record of the company's accounting transactions
Accrual	An estimate of an expense already incurred but not yet invoiced or paid
Journal entry	Balanced accounting record: total debits equal total credits
True-up	Comparison of estimate with later actual invoice and correction/explanation of the difference
Materiality	Dollar/risk threshold above which human approval is required
Workpaper	Evidence, calculation, and rationale supporting an accounting decision
Vendor obligation	Our canonical case file for one expected vendor cost during a defined service period

Central example

Northstar Analytics uses DataForge's data platform in December.

- DataForge service was used from December 1–31.
- The December invoice has not arrived at December close.
- Contract/prior invoices indicate a monthly fee around \$50,000.
- TrueUp Close prepares a December accrual.
- In January, the actual invoice arrives for \$52,500.
- The system matches it to the December obligation, finds a \$2,500 under-accrual, and identifies a 5% contract price escalation.
- It updates close/AP/cash/audit work products and proposes a policy requiring effective-price validation for similar contracts.

Accounting mechanics used in the demo

At December close, if services were received but invoice is missing:

Debit:	Data Services Expense	\$50,000
Credit:	Accrued Expenses Liability	\$50,000

On January 1, the accrual can reverse automatically:

Debit:	Accrued Expenses Liability	\$50,000
Credit:	Data Services Expense	\$50,000

When the January invoice arrives:

Debit:	Data Services Expense	\$52,500
Credit:	Accounts Payable	\$52,500

Net January expense impact after the reversal is \$2,500. That is the true-up caused by the original estimate being low.

3. Scope: what we automate and what we do not

In scope: one connected CFO workstream

Workflow	What TrueUp Close does
Pre-close vendor-spend intake	Detects recurring/expected vendor obligations and checks whether invoices have arrived
Evidence collection	Pulls contracts, POs, usage/service records, prior invoices, AP state, and ledger context
Invoice validation	Flags duplicate, late, mismatched, multi-period, or unsupported vendor invoices

Workflow	What TrueUp Close does
Accrual preparation	Estimates unbilled services with approved deterministic estimators; creates workpaper and draft entry
Uncertainty resolution	Selects the best internal or external person/system to ask for missing information
Review and approval	Routes material, conflicting, or low-confidence cases to reviewer/Controller
Invoice-to-accrual reconciliation	Matches later invoice, computes true-up, prepares explanation and correction path
Downstream handoffs	Produces AP-ready, close-ready, cash-ready, FP&A-ready, and audit-ready outputs
Policy verification	Blocks actions that violate encoded finance procedures
Governed learning	Proposes, backtests, approves, activates, and revokes scoped policies

Out of scope

Not building	Why
Full ERP/general ledger	TrueUp Close runs on top of an ERP; it uses a small simulated ledger only
Payment execution	No ACH, wire, card, or bank payment release
Full AP automation	No universal invoice OCR, vendor onboarding, payment batches, or supplier master-data management
AR/collections/cash application	Different CFO workstream
Bank reconciliation	We reconcile invoice-to-accrual, not bank statements to all GL activity
Full company close	No payroll, tax, revenue recognition, fixed assets, leases, intercompany, or consolidation
Full cash forecasting	Only provides vendor-payment forecast inputs
Autonomous accounting-policy decisions	Human Controller retains policy and approval authority
Production systems or live outreach	Simulated integrations, inbox, and responses for the hackathon

Product boundary in one line

We automate the vendor-spend close workstream deeply, then hand structured outputs to the rest of the finance organization.

4. Canonical data model and state machine

Canonical object: Vendor Obligation

A `VendorObligation` is the shared representation of one expected vendor cost for one service period.

```
VendorObligation
- obligation_id
```

- vendor_id
- legal_entity_id
- service_period_start / service_period_end
- close_period
- category / cost_center
- contract_id / po_id
- expected_bill_date
- invoice_status
- service_received_status
- evidence_ids
- estimator_method
- estimated_amount
- confidence_score
- risk_score
- accrual_entry_id
- invoice_id
- true_up_amount
- root_cause
- status
- policy_version_used
- approval_history
- outreach_history

Supporting objects

Vendor
Contract
PurchaseOrder
UsageRecord
ServiceReceipt
Invoice
AccrualWorkpaper
JournalEntry
InvoiceMatch
TrueUpAdjustment
OutreachTask
OutreachResponse
Policy
PolicyEvaluation
Approval
AuditFinding
CashForecastLine
VarianceExplanation

Obligation state machine

DISCOVERED
-> IN_EVIDENCE_GATHERING
-> INVOICE_VALIDATED
-> ACCRUAL_CANDIDATE
-> PENDING_REVIEW
-> ACCRUED
-> INVOICE_RECEIVED

```
-> MATCHED
-> TRUE_UP_REQUIRED
-> RECONCILED
```

```
Any state -> OUTREACH_PENDING
Any unsafe/ambiguous state -> ESCALATED
Any duplicate/invalid case -> HOLD_DO_NOT_POST
```

Required invariant

Every downstream record references the same obligation_id. This is how the same transaction remains consistent across close, AP, cash, reporting, and audit.

5. End-to-end workflow

Step 1 — Discover expected vendor spend

Trigger: Month-end close begins or system sees a new invoice/change event.

The system identifies vendor obligations using:

- recurring invoice history;
- contract billing schedule;
- active purchase orders;
- usage/service records;
- prior accrual history.

Question: Did the company receive a vendor service this month, and is it fully accounted for?

Step 2 — Gather evidence

The Evidence Agent retrieves:

- active contract and pricing schedule;
- PO and change orders;
- current-month service/usage evidence;
- prior invoices;
- AP invoice queue;
- existing GL entries and prior accruals;
- company policies and close deadlines.

It returns structured facts with citations to source records.

Step 3 — Classify the obligation

Classification	Action
Valid invoice exists and matches evidence	Mark invoice close-ready
Invoice exists but is mismatched/duplicate/ambiguous	Open exception and gather evidence
No invoice, service was received, evidence sufficient	Prepare accrual candidate
No invoice, service evidence unclear	Targeted outreach or escalation
No invoice, service not received	No accrual; document reason
Invoice spans periods	Allocate by service period and review as needed

Step 4 — Estimate or validate amount

Approved deterministic estimation methods:

Method	Required inputs
Fixed contract fee	Active contract price + service period
Usage × contracted rate	Usage record + valid rate card
Prorated fee	Contract fee + service dates
PO-based estimate	Approved PO + evidence of service received
Historical run rate	Prior invoices; only low-risk fallback with lower confidence

The LLM may identify the relevant contract term or select an allowed method. Python code calculates the amount.

Step 5 — Verify procedures and choose next action

Every proposed action passes through the Policy Verification Engine.

Possible results:

```
PASS: prepare draft workpaper / route normally
REVIEW_REQUIRED: obtain Controller or reviewer approval
OUTREACH_REQUIRED: request a missing fact from best owner
BLOCK: do not post; evidence/policy violation
ESCALATE: human must choose accounting treatment
```

Step 6 — Resolve uncertainty with targeted outreach

The agent selects the lowest-cost person/system that can answer the missing question.

Missing fact	Target	Example request
Was a service received?	Operational/service owner	“Please confirm whether DataForge was active for Dec 1–31.”

Missing fact	Target	Example request
Did scope/rate change?	Requester or PO owner	“Was there a December change order or usage expansion?”
Where is expected invoice?	Vendor billing contact	“Please provide the December invoice or confirm expected issue date.”
Does AP already have invoice/credit?	AP specialist/system	“Is a DataForge invoice or credit memo pending in AP?”
Which treatment is allowed?	Accounting manager/Controller	“Material case has conflicting evidence; choose treatment.”

The demo uses a simulated inbox/queue and fixture responses. An unanswered request is not evidence.

Step 7 — Create close work product

For a supported accrual, generate:

- accrual workpaper;
- cited evidence pack;
- deterministic calculation trace;
- confidence/risk explanation;
- draft journal entry;
- policy-verification result;
- reviewer/Controller task if required.

Step 8 — Invoice arrival and true-up

When the actual invoice arrives:

1. Match it to a prior obligation/accrual.
2. Allocate service periods if necessary.
3. Compute $\text{true_up} = \text{actual_invoice_amount} - \text{prior_accrual_amount}$.
4. Diagnose the difference with evidence.
5. Prepare reconciliation/adjustment workpaper.
6. Update downstream outputs.
7. Feed confirmed outcome into learning.

Step 9 — Hand off downstream outputs

Recipient	Output
Close accountant	Close status, approved entries, open exceptions, evidence workpapers
AP	Invoice/accrual match, coding, service-period allocation, exception state

Recipient	Output
Treasury	Expected payment amount, due date, currency, forecast delta
FP&A	Spend variance and root cause
Auditor	Evidence chain, calculation, policy check, approvals, true-up history

6. Agents and responsibilities

Agents communicate through structured records, not free-form chatbot debate.

Agent	Goal	Inputs	Outputs	Cannot do
Evidence Agent	Build source-backed fact set	Contracts, PO, invoice, usage, AP, GL	Extracted facts + citations	Invent missing facts
Obligation Agent	Discover and classify expected vendor obligation	Vendor history, contracts, AP, service data	Obligation record + state	Post accounting entry
Estimation Agent	Propose permitted amount/method	Structured evidence + estimator policies	Calculation request/workpaper	Generate unconstrained amount
Verification Engine	Enforce procedures	Proposed action + policy + state	Permit/block/review/outreach decision	Interpret documents or override policy
Outreach Agent	Resolve missing evidence efficiently	Missing-fact type, org/vendor directory	Drafted request + assigned recipient	Treat no response as confirmation
Reviewer Agent	Independently check support and policy	Workpaper, evidence, verifier result	Review finding / return / approval recommendation	Override protected controls
Reconciliation Agent	Match invoice and calculate true-up	Invoice, obligation, accrual, ledger	Match, variance, root cause request	Decide unusual accounting policy alone
Learning Agent	Turn confirmed errors into candidate policies	Invoice outcomes, human resolutions, outreach responses	Candidate policy + replay report	Activate policy independently
Auditor Agent	Re-perform and document controls	All linked objects	Audit result/workpaper	Change financial state
Human Controller	Own material judgment and approval	Reviewer pack, exceptions, policy proposals	Approval/rejection/override rationale	N/A

7. Policy verification / formal-control layer

Honest claim

We do not formally prove an LLM’s interpretation is always correct. We verify that every consequential workflow action satisfies encoded company controls.

Action model

Every agent submits a typed action proposal:

```
{
  "action_type": "POST_ACCRUAL",
  "obligation_id": "OBL-DATAFORGE-DEC",
  "period": "2026-12",
  "amount": 50000,
  "currency": "USD",
  "accounts": {
    "debit": "Data Services Expense",
    "credit": "Accrued Expenses"
  },
  "calculation_method": "FIXED_CONTRACT_FEE",
  "evidence_ids": ["CTR-482", "PO-1029", "USAGE-DEC-DF"],
  "confidence": 0.91,
  "policy_version": "1.0"
}
```

Key policy invariants

Invariant	Verification
Entry balances	Debits equal credits
Period is open	No action posts to a closed period
Service receipt supported	Required service/usage/owner evidence exists
Required identifiers present	Vendor, entity, cost center, period, account, and obligation exist
No duplicate accrual	No active accrual already exists for same obligation/period
Allowed estimation method	Method belongs to policy allow-list
Evidence trace is complete	All required source IDs attached
Entity/account compatibility	Contract/PO/entity/account relationship is valid
Materiality approval	Amount/risk threshold determines human approval requirement
Uncertainty cannot post	Low confidence or conflicting evidence produces outreach/review/escalation
No silent policy weakening	Learning cannot reduce controls or activate without approval
Every action traceable	Store evidence IDs, policy version, actor, verifier result, timestamp

Example verifier behavior

Agent proposes \$50,000 accrual.

- ✓ Open period
- ✓ Balanced draft entry
- ✓ Service evidence exists
- ✓ Contract/PO exists
- ✓ No duplicate accrual
- ✗ Price schedule missing despite contract escalator clause
- ✗ Amount exceeds Controller threshold

Result: BLOCK_POSTING
Next action: Request effective pricing schedule from vendor contact.
If unresolved by close deadline: Controller review.

8. Human-in-the-loop and escalation model

Principle

Humans should handle genuine judgment, not routine data gathering. The system should route cases based on evidence quality, materiality, risk, and deadline.

Confidence/risk routing

Condition	System behavior
High confidence + low risk	Prepare workpaper automatically; batch review if policy allows
High confidence + material amount	Prepare workpaper; require Controller approval before posting
Medium confidence	Ask targeted internal/external question, then reassess
Low confidence or conflicting evidence	Block posting; escalate to reviewer/Controller
Unknown / no evidence	Do not guess; create exception and ownership task

Escalation ladder

0. Resolve automatically from verified evidence
1. Ask operational/service owner
2. Ask requester or PO owner
3. Ask AP specialist/system
4. Ask vendor billing contact
5. Escalate to Accounting Manager / Controller

This is not a rigid sequence. The agent chooses the best source for the missing fact. For example, only the vendor may know when it will issue an invoice, while only the service owner may know whether work was received.

9. Governed self-improvement

Feedback sources

Signal	What it teaches
Later actual invoice	Whether the prior estimate was accurate
Invoice-to-accrual variance	Quantitative size/direction of error
Human review correction	How a finance expert resolved an edge case
Vendor response	Invoice timing, billing cadence, effective price, service details
Operational-owner response	Whether service was received and which period/cost center applies
Audit finding	Whether evidence/control procedure was insufficient

Learning lifecycle

Confirmed outcome

- Root-cause classification
- Candidate typed policy
- Historical replay / regression test
- Controller approval
- Provisional activation
- Monitor future cases
- Promote, weaken, revoke, or supersede

Example learned policy

```
{
  "policy_type": "VERIFY_EFFECTIVE_CONTRACT_PRICE",
  "trigger": "RECURRING_VENDOR_ACCRUAL",
  "scope": {
    "vendor_category": "SAAS_OR_DATA",
    "contract_feature": "SCHEDULED_ESCALATOR"
  },
  "required_evidence": [
    "pricing_schedule",
    "effective_date",
    "service_period"
  ],
  "behavior_change": "retrieve_and_validate_current_rate_before_estimation",
  "autonomy_limit": "controller_review_if_material",
  "status": "CANDIDATE"
}
```

Required proof of learning

A later held-out vendor should contain a similar effective-price problem. The frozen baseline should miss it; the policy-enabled agent should retrieve the pricing schedule, use the right calculation, or escalate if evidence remains inadequate.

10. Data fixtures

Simulated company

Northstar Analytics: B2B software company with 12–20 vendors and three close periods.

Required documents/data

- Contracts: PDFs/text with pricing schedules, escalators, renewals, and ambiguities
- POs: vendor, owner, cost center, authorized amount
- Usage/service receipts: cloud/data/consulting/service evidence
- Historical invoices: recurring patterns and timing
- Current invoices: some valid, late, duplicate, mismatched, and multi-period
- AP queue: invoice status and pending records
- Simulated GL: posted accruals, reversals, and invoices
- Org directory: operational owner, requester, AP specialist, Controller, vendor contact
- Close policy manual: thresholds and rules
- Simulated inbox responses: confirmations, missing invoice, no response, conflicting facts

Required cases

Case	Purpose
Exact recurring invoice match	Happy path; validates standard processing
Missing invoice, high-confidence fixed contract	Supported accrual path
DataForge 5% escalator	Main true-up and learning story
Missing service confirmation	Outreach to operational owner
Missing invoice response from vendor	External outreach changes action
Duplicate invoice	Control/fraud-prevention style exception
Multi-period invoice	Service-period allocation reasoning
Unknown/conflicting evidence	Demonstrates safe escalation rather than hallucination

11. Architecture and UI

Suggested stack

- Backend: Python + FastAPI
- Models/schemas: Pydantic
- Database: SQLite for hackathon speed; Postgres only if already ready
- Calculations/policy verifier: deterministic Python
- LLM: document extraction, evidence reasoning, root-cause explanation, outreach drafting, structured action proposals
- Frontend: Next.js/React if team is fast; Streamlit is acceptable if polish/time requires

Essential backend services

/ingest	load fixtures/documents
/obligations	discover/list vendor obligations
/evidence	retrieve and cite evidence
/estimate	run deterministic estimator
/verify-action	policy verification
/outreach	create/update outreach tasks
/review	reviewer and Controller decisions
/accrual	create draft accrual/entry
/invoice-match	match invoice to obligation
/true-up	calculate variance and root cause
/policies	propose/replay/approve policies
/audit	produce audit packet
/evaluate	run baseline vs learner evaluation

Essential UI screens

1. Close Command Center

- obligation inventory by status;
- close progress;
- open exceptions;
- approval queue;
- materiality and risk summary.

2. Vendor Obligation Workpaper

- evidence timeline;
- extracted contract/PO/usage facts;
- estimate/calculation trace;
- policy-verifier checklist;
- reviewer/Controller decision.

3. Outreach and Resolution Queue

- why system is uncertain;
- selected recipient;
- drafted request;
- simulated response;
- next action after response.

4. Invoice and True-Up View

- actual invoice vs prior accrual;
- difference and service-period allocation;
- root-cause evidence;
- close/AP/cash impact.

5. Learning and Controls View

- candidate policy;
- replay result;
- Controller approval;
- active policy version;
- baseline-versus-learned results.

6. Audit Packet

- evidence lineage;
- calculation re-performance;
- action/approval/policy log;
- pass/fail control findings.

12. Evaluation and demo

Metrics

Metric	Why it matters
Accrual mean absolute percentage error	Estimate quality versus later actual invoice
Total absolute dollar error	Financial materiality of mistakes
Material-miss rate	High-risk estimate failures
Correct invoice/accrual match rate	Reconciliation capability
Root-cause diagnosis accuracy	Whether explanations identify true seeded cause
Correct escalation-routing rate	Whether the right person is contacted
Unnecessary human-review rate	Efficiency / HITL quality

Metric	Why it matters
Unsupported autonomous postings	Trust metric; target is zero
Policy regression count	Number of prior correct decisions made worse by learning; target is zero
Held-out improvement	Whether approved learning improves unseen vendor/month cases

Baselines

1. Historical trailing-average estimator.
2. Static rules agent with all initial rules but no learning.
3. Policy-enabled TrueUp Close learner.

Three-minute demo story

1. Show close command center: a recurring DataForge invoice is missing.
2. Open the obligation: system collected contract, PO, usage, invoice history, and ledger context.
3. Show proposed accrual, formula, evidence, and verifier decision.
4. Demonstrate uncertainty: contract contains a price escalator but effective price is unresolved.
5. Show targeted outreach request to vendor/contact or owner and resulting simulated evidence.
6. Show Controller review only because materiality/risk requires it.
7. Advance time: actual invoice arrives for \$52,500 versus \$50,000 accrual.
8. Show match, true-up, root cause, AP/cash/audit downstream outputs.
9. Show candidate policy, replay test, and Controller approval.
10. Run held-out vendor: baseline misses escalation; learned agent retrieves effective pricing and gets correct result/escalates safely.
11. End with metrics and immutable action/policy trace.

Demo claims to make

- “We do not replace the ERP or Controller.”
- “We automate the vendor-spend close workstream and provide clean handoffs to the rest of finance.”
- “We only escalate genuine uncertainty.”
- “The model cannot bypass encoded company procedures.”
- “Actual invoices, approved decisions, and outreach responses create governed learning.”

Claims not to make

- “We formally prove the LLM is always correct.”
- “We fully automate a company’s accounting function.”
- “We autonomously change accounting policy.”
- “We execute payments or production journal entries.”

13. Definition of done

The MVP is complete only if the team can demonstrate this exact sequence end to end:

1. System identifies an expected vendor bill missing at close.
2. It creates one canonical vendor obligation.
3. It gathers contract, PO, usage, historic invoice, AP, and GL evidence.
4. It calculates a deterministic accrual and generates a workpaper.
5. It sends the proposed action through the Policy Verification Engine.
6. The verifier blocks or routes an unsafe/uncertain case correctly.
7. The system asks the correct human/vendor contact for missing information in the simulated inbox.
8. A response changes the system’s next action.
9. A Controller approves the material supported case.
10. A simulated accrual entry is created in the ledger.
11. A later invoice is matched to the original obligation.
12. The system calculates and explains a true-up from evidence.
13. It generates AP-ready, cash-ready, variance-ready, and audit-ready outputs.
14. It proposes a candidate learning policy.
15. The policy is replay-tested and requires Controller approval.
16. The approved policy improves or safely escalates a held-out future case.
17. Every consequential action shows evidence IDs, policy version, verifier result, and approval history.

Final team rule

Every feature must either gather evidence, resolve uncertainty, create a controlled close work product, produce a useful downstream handoff, enforce a company procedure, or prove learning on a future case. Otherwise, cut it.